YEAR 10 ICT PROGRAMMING



NAME: _____

CLASS: _____

What is programming?

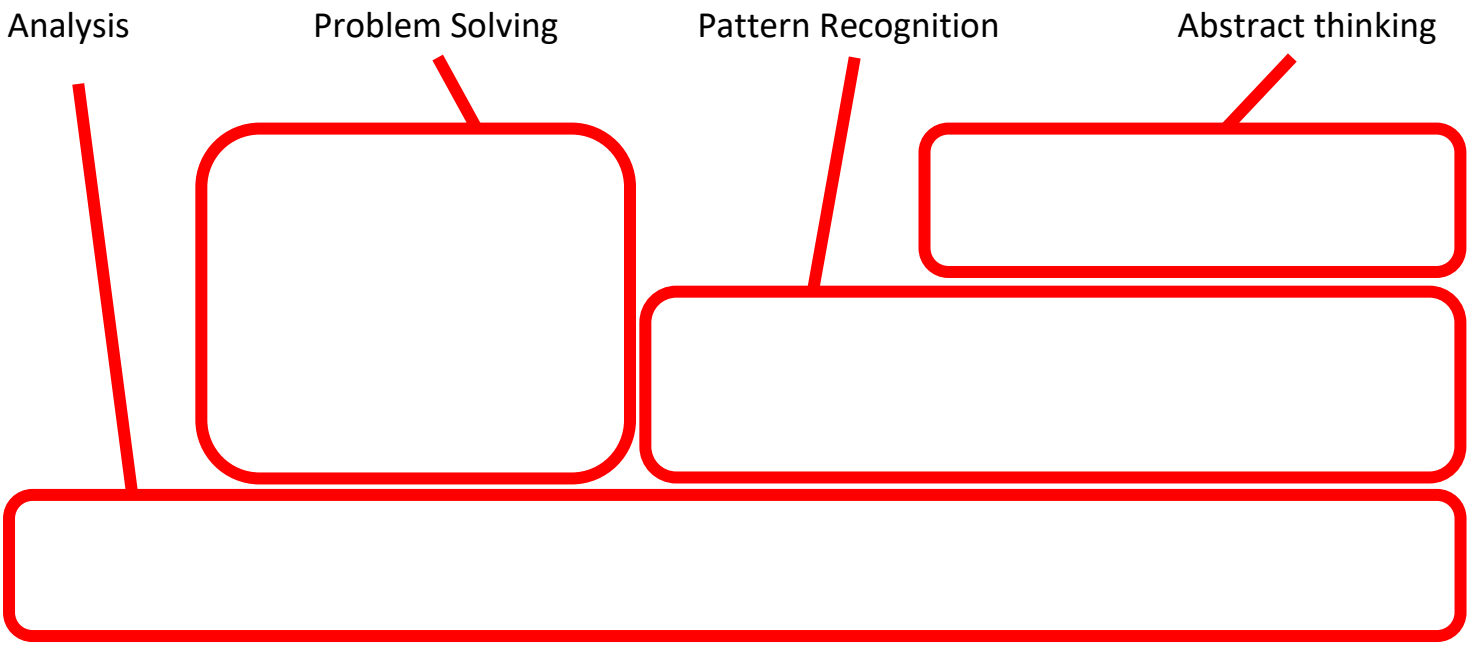
Programming is often called ‘coding’, as it involves writing a language that computers can understand. This allows us to develop devices, apps and technology in our own way.

A **programmer** is a person who writes code for a device and is also involved in testing the code to ensure it is fully functional.

Programming Languages

The Jobs in Programming

Learning programming offers a huge number of opportunities in terms of career paths. Not only can you get a position directly related to IT, but you can also open up opportunities to go into Medicine, Marketing, and even the army! This is because there are a number of skills being developed when you are learning a programming language. The top skills could be suggested to be:



Think of all the different times you have needed to analyse a piece of text in English or solve a problem in Mathematics. The reason for this is so that these underlying skills can be improved as they encompass a huge number of jobs and tasks you will need to have in your working life. Computer Science directly teaches you these important skills, and it is due to having these skills is why programmers in particular have such a high salary and are known to be on much higher wages across the globe.

Programming Experience

In school, you will already have experience with some applications which will help you with progressing onto what computer scientists call a “programming language”. So far, you may have experienced programs such as Scratch, or even a Microcontroller like the BBC Microbit. You may also have learnt some syntax when dealing with HTML Web Design. These, amongst others, have led you to already have some knowledge of the underlying concepts of programming.

Programming Concepts

There are three ICT concepts that you have learnt previously – however you may not have known that you have been learning programming concepts as they may have been taught in different subjects as well as in ICT class.

These include:

SEQUENCING

- ✓ This is the flow or logical order to instructions. An algorithm is a sequence of instructions which flow one after the other.

SELECTION

- ✓ Selection is where a program decides if a line or group of lines of code need to be executed (ran) or not.
- ✓ This is achieved through the use of **IF** statements usually. For example, **if** a condition is met, run this code, **otherwise**, run this code.

REPETITION

- ✓ Also known as iteration, which is the process of repeting a sequence of steps.
- ✓ It uses loops so that code can be repeated. It comes in a few formats:
    - Count controlled loop (a piece of code loops a specified number of times)
    - Condition Controlled loop (when it repeats an unknown amount of times before it finishes looping)
    - Infinite loop (loops forever, such as traffic lights)

What programming Language do we Use?

There are lots of programming languages to choose from, with some of the famous ones being Python, Java and C#. In ERGS, we learn C# as it is versatile, relatively easy to learn and comes with a lot of documentation as it was written by Microsoft.

It is a high level language, which means it is very close to human readable English. Other languages, like Assembly Language, would be known as a low level language, meaning it is very close to being computer readable (binary).

As we know computers only understand binary, and do not even understand a programming language, we need to have a way of converting from our programming

language to the binary code computer *do* understand. This process is done by the computer software we use, with the process being called **compiling**.

The C# (C Sharp) Programming Language

C# is an elegant object-orientated language that enables developers to build a variety of secure and robust applications that run on the .NET Framework.

For example, the mobile game Pokemon Go and the Stack Overflow website were built with frameworks that can be run with C# (Unity and ASP.NET, respectively).

You can use C# to create Windows applications, Web services, mobile applications, and much more. It is a very popular programming language, being in the top 10 most used throughout the world because it is used in nearly every Microsoft product in the world.

> Don't worry about this term too much. It basically means that for a variable, you must declare its data type before it can be used. The tutorials will go into this later.

C# has a lot of benefits, as it is a strongly typed language.

C# is used to make interactive websites, mobile apps, video games, augmented and virtual reality (AR and VR), back-end services, and desktop applications.

Benefits of Using a strongly typed language are:

- ✓ Built in **documentation** – which helps you know how the data types work
- ✓ Increased **readability**, helping programmers to work in teams
- ✓ Use of **intellisense** – Which is a bit like predictive text when writing code
- ✓ Assistance in **debugging** – the language knows what can and can't happen with a data type before you even try to run your code.

How to Write Code

Programmers wouldn't use just any old application to write their code – they would use specialised software to help with the speed and efficiency of writing programs. Whilst several basic programming packages exist (such as notepad), it would be

common to use a 'proper' piece of software like **Microsoft Visual Studio**. This is because it allows us to view, write, run and test code.

Coding software is often called an Integrated Development Environment, or IDE, as it integrates all of the features that help to write code in one place.



An Integrated Development Environment is useful because:

- ✓ Allows programmer to enter and edit code
- ✓ Provides access to objects, methods, properties and events at design time

Several features include:

- ✓ Line numbering: helps programmers distinguish between lines in long programs. Also helps compiler reference location of errors in code
- ✓ Colour coding of keywords: Also different coloured underlines – red underline, for example, displays a syntax error
- ✓ Collapsible code sections: Makes viewing large programs easier
- ✓ Clipboard (for copy+paste)

TASK

Create an Account on Code Academy.

Start the tutorial lessons called **Learn C#**

This should give you an introduction to C#. Follow the instructions to take you through C# coding, including the language and some of the syntax (spelling/punctuation/grammar) of C#

Writing Text to the Console

Text can be written out to the display using the **Console.WriteLine()** method.

This method should have an input inside the parentheses (or more commonly known as brackets). Inside the brackets, you should put speech marks and the text you want to display. For example;

Console.WriteLine("Mr Butler");

Will print out

```
Mr Butler
```

TASK

Go to the website https://dotnetfiddle.net/

Have a go at creating a program that outputs:

- Your name
- Your Favourite Sport or Hobby
- Your Favourite Subject in school

Take a screenshot of the completed code (with output) and paste it below.

Getting Input from the User

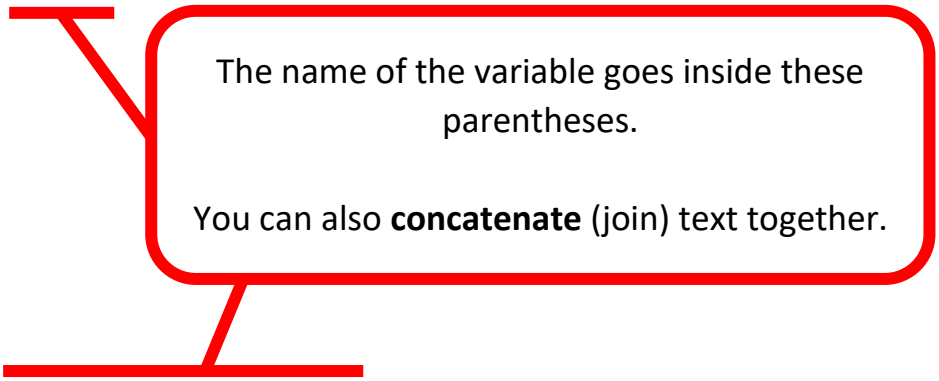
When programming, it is really important to have the ability to take some input from the user – usually text, numbers, or other data.

This uses a method called **Console.ReadLine();** which allows the program to take in a line of text. You can then **assign** this value to a variable as shown below.

```
String myText = Console.ReadLine();
```

You can then output this value using the Console.WriteLine() method from earlier.

```
Console.WriteLine(myText);
```




```
Console.WriteLine("My Name is " + myText);
```

Commenting Code

Now that our programs are going to start getting longer, it's important we are able to follow, at a glance, what our program is doing at a particular time.

Comments, then, are the ability to provide programmers with little annotations about code lines or sections of code to know exactly what the program is doing or a reason for a piece of code.

They are essential in programming. It is essential to have the ability to tell a computer to ignore part of a program as and when it is being developed.

Text written in a program but not run by the computer is called a comment. In C#, anything after a **//** or between **/*** and ***/** is a comment.

Comments can:

- Provide context for why something is written the way it is
- Help other people reading the code understand it faster
- Ignore a line of code and see how a program will run without it.

Developers tend to use // for short one line comments where as if a comment needs to be multiple lines, you should use /* and */

In most development environments, comments are automatically made to be a different colour, and as they don't need to be understood by a computer, can be written in plain English. This makes it great because if you ever needed to come back to a section of code after a period of time, you could quickly read the comments and know what the code was doing.

In Visual Studio, comments appear in green.

```
// this is an example of a single line comment


/*
        This is an example of a comment
        that can be several lines long
*/
```

Data Types and Variables

When we write programs, we're telling the computer how to process pieces of information, like the numbers in a calculation, or printing text to the screen.

Languages like C# tell a computer about the type of data in its program using *data types*. Data types represent the different types of information that we can use in our programs and how they should be used.

Without data types, computers would try and perform processes that are impossible, like squaring a piece of text or capitalizing a number. That's how we get bugs!

C# is *strongly-typed*, so it requires us to specify the data types that we're using. It is also *statically-typed*, which means it will check that we used the correct types before the program even runs. Both language features are important because they help write scalable code with fewer bugs.

Using types will come up in several different places when learning C#.

Data types tell us a few things about a piece of data, like:

- How it can be stored
- What operations we can perform with it
- Different methods it can be used with

Data types are present in all programming languages, but are particularly important in C#. That's because C# is known as a strongly-typed language—it requires that the programmer *specify the data type of every value and expression*.

Type	Description	Size (bytes)	Used For	Range
int	Whole numbers	4	Whole Number	-2, 147,483,648 To 2, 147, 483, 647
long	Whole numbers (bigger range)	8		-9, 223,372,036, 854, 775, 808 to 9,223,372,036, 8,54,775,807
float	floating- point numbers	4	Numbers with Decimal points	+/-3.4x10^38
double	Double precision (more accurate) FP numbers	8		+/-1.7 x 10^308
decimal	Monetary values	16		28 significant figures
char	Single character	2	Text	N/A
bool	Boolean	1	Boolean	True or False
DateTime	Moments in time	8	Date/Time	0:0:00 on 01/01/0001 to 23:59:59 on 12/31/9999
string	Sequence of characters	2 per character	Text	N/A

Note: You don't need to remember the range of these data types, but you should only ever use a data type of suitable size!

The most common data types we would use are:

Int double char string bool

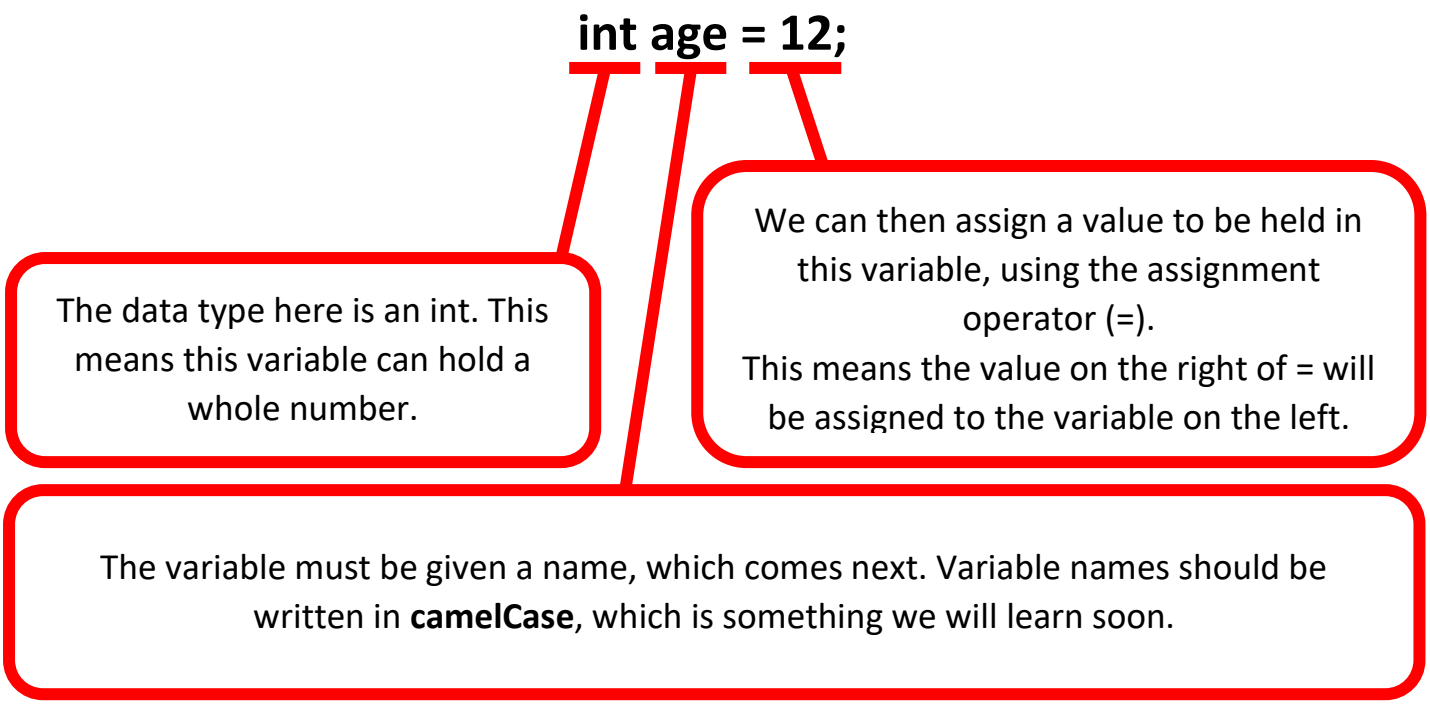
Variables

Programs typically use data to perform tasks. Creating a **variable** reserves a memory location, or a space in memory, for storing values. It is called **variable** because the information stored in that location can be changed when the program is running.

To use a variable, it must first be declared by specifying a **name** and **data type**.

A variable name, also called an **identifier**, can contain letters, numbers and the underscore character and must start with a letter or underscore**.** Although the name of a variable can be any set of letters and numbers, the best identifier is **descriptive** of the data it will contain. This is very important in order to create a clear, understandable and readable code!

A variable is declared by its data type first, then variable name.



For example, **firstName** and **lastName** are good descriptive variable names, while **abc** and **xyz** are not.

A line of code that completes an action is called a statement. Each statement in C# must end with a **semicolon**.

Remember that you need to declare the variable before using it.

Handling Errors

When you program, you'll come across a lot of errors. And that's ok! And when you're first starting to learn a strongly-typed language, they can be pretty common.

These can be caused for a number of reasons:

- Forgetting to specify a data type
- Using a variable without declaring it first
- Using the wrong data type
- How variables are named – You should use camel case!
- There are a few words you cannot use for variable names – these are known as **reserved keywords**.

TASK

There are 5 errors in this code snippet. Copy out this snippet and highlight/circle the errors.

Annotate how or what the correct solution should be!

```
1   using System;
2
3   namespace BugSquasher
4   {
5     class Program
6     {
7       static void Main(string[] args)
8       {
9         int number = 38498.3222;
10
11        dinosaur = "Barney";
12
13        double lock = 293.000;
14
15        bool is.yes = true;
16
17        string band = "The Low Anthem"
18
19      }
20    }
21  }
```

/5

Converting Data Types

Because variables have to be strictly typed, there may be some circumstances where we want to change the type of data a variable is storing.

This strategy is known as **data type conversion**.

There are two types of conversion:

Implicit conversion:

- Happens automatically as no data is lost in the conversion. For example, you can convert an *int* to a *double* as a double can hold more data anyway

Explicit conversion:

- Requires a cast operator, so if we wanted to convert a *double* to an *int*, we could use the operator (int).

One of the most common ways of converting data types is to use the **Convert.To…** method calls.

For example,

```
Int myValue = Convert.ToInt32("42");
```

This variable can hold an integer data type

As this value is in double quotation marks, the computer thinks this is a piece of text (string)

This Convert.ToInt32 is a predefined method which tells the computer to convert the string input into an integer (For now, let's ignore the 32 at the end of this)

TASK

Let's convert a string input and store in an integer variable called age.

TASK: Create a program that asks a user for their name, and then their age. Store their name in a variable called myName and their age in a variable called myAge.

Arithmetic Operators

When writing programs, we often need to perform calculations on different values and variables. These are particularly useful when dealing with numerical data types.

We will be using different **arithmetic operators**. These include:

- Addition +
- Subtraction –
- Multiplication *
- Division /

We can use these symbols to perform operations on numbers and create new values.

```
int answer = 24 + 18;
Console.WriteLine(answer);
// prints 42
```

It's important to keep an eye out on the different data types when we are using arithmetic operators, because if we use two different data types in our operation, the answer will take the bigger data type.

```
Console.WriteLine(5 / 3);
Console.WriteLine(5 / 3.0);
// prints 1
// prints 1.66667
```

The first operation that we log uses two ints. While 3 doesn't go into 5 evenly, we are still left with an int whole number. In the second operation, we use an int and a double, so the final result is a double.

Operator Shortcuts

Often we need to update a variable in our program. We can do so by modifying that variable using an arithmetic expression, then re-saving it to the same variable name:

```
Int apple = 0;
apple = apple + 1;
Console.Write(apple);
// prints 1
```

```
Int apple = 0;
Apple++;
Console.Write(apple);
// prints 1
```

As programmers, we are always looking for shortcuts. So we can condense the same program above using the shorthand ++

The combined addition signs represent the idea of *incrementing by one*. We can do the same with subtraction symbol – to decrease a variable value by 1.

Operators: Modulus %

This operator returns the remainder after a division. It uses the percentage symbol, but it's important to remember it has a different meaning in this context. It's very useful as it lets us know if a number 'fits' into a larger number, or if there will be a remainder.

```
Int remainder = 4 % 3;
Console.WriteLine(remainder);
// prints 1
```

IF Statements

A really common function in programming is the ability to ask questions and to do different things depending on the outcome.

To do this, we use an **IF** statement.

Operator	Meaning
<	Less than
<=	Less than or equal to
>	Greater than
>=	Greater than or equal to
==	Equal to
!=	Not equal

An example in C# would be:

```
Console.WriteLine("Please enter a number greater than 0");
int myNumber;
myNumber = Console.ReadLine();
if(myNumber < 0){
        Console.WriteLine("Number is not bigger than zero!");
}
```

Else Clause

You have often found in programs like scratch and Microbits, were the IF statement is linked to an ELSE statement. This else option runs if the condition is false.

An optional **else** clause can be specified to execute a block of code when the condition in the **if** statement evaluates to **false**.

```
if(condition)
{
  // statements
}
Else
{
  // statements
}
```

```
int mark = 85;
if(mark < 50)
{
  Console.WriteLine("You failed.");
}
Else
{
   Console.WriteLine("You passed.");
}
```

You can even nest if statements inside of another if statement.

Loops and Arrays

Another great data structure is the ability to repeat code. We can use a **loop** to repeat code without having to type it out over and over again. There are two types of loops awe can use:

- Conditional loops (Condition Controlled Loop)
- Unconditional loops (Count Controlled Loop)

Count Controlled Loops

Before you 'enter' the loop, you know exactly how many times you will repeat the code.

For example, in a 100m race track, you may be asked to run the **loop 10 times** to make a 1000m race



Condition Controlled Loops

These are loops which until a condition, value or input changes, will run an unknown number of times.

Lets keep to a running track example, “run till you are out of breath” will have an unknown number of laps.

TASK

Create a table in Microsoft Word that defines the difference between a **count controlled** and a **condition controlled** loop.

- What programming structures are used to create these in C#?

Summarise these differences in a table.

At the bottom of your document, state what is similar between these loops

Objects and Instances

Object-oriented programming is based on the concept of **objects**, and how objects can interact with one another.

In object-oriented programming data structures, or objects are defined, each with its own properties or attributes. Each object can also contain its own procedures or methods.

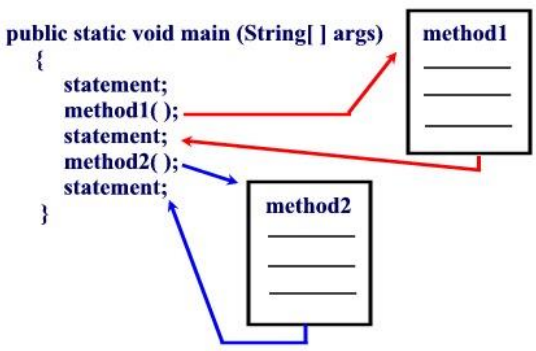
Objects are used as they allow us to create methods which allow us to reuse code we have written for one object and use it in another without the need for rewriting it all.

Methods

Methods are small groups of instructions which run when they are ‘called’ in the code.

The code within each method will only execute when the method has been called and will execute in sequence.

When calling methods, brackets are used to include any parameters for the method to use.



Basic Methods

This is a method. We call it by using the **methodName** followed by brackets **()**

```
using System;

public class Program
{
    public static void Main()
    {
        Console.WriteLine("Let's say hello in C#");
        sayHello();
    }

    public static void sayHello(){
        Console.WriteLine("Hello you!");
    }
}
```

Methods with Parameters (inputs)

If a method has the ability to accept a variable, you can add this into the brackets.

When creating the method, you <u>must</u> state what the data type is being expected to be sent

```
using System;

public class Program
{
    public static void Main()
    {
        Console.WriteLine("Let's say hello in C#");
        sayHello("Mr Butler");
    }

    public static void sayHello(string name){
        Console.WriteLine("Hello " + name);
    }
}
```

This is a more complex example of a method. It sends two values into the method.

The method then adds them.

The 'answer' is then returned to the main method.

```
public class Program
{
    public static void Main()
    {
        int x = 6;
        int y = 7;
        int answer = 0;
        answer = addTwoNumbers(x, y);
        Console.WriteLine("The answer is " + answer);
    }

    public static int addTwoNumbers(int num1, int num2){
        int solution = num1 + num2;
        return solution;
    }
}
```